

CH4 – Clip 8.

쿠버네티스 실습



01

환경 구축

03

쿠버네티스 주요 구성 요소

02

쿠버네티스와 아키텍처

03

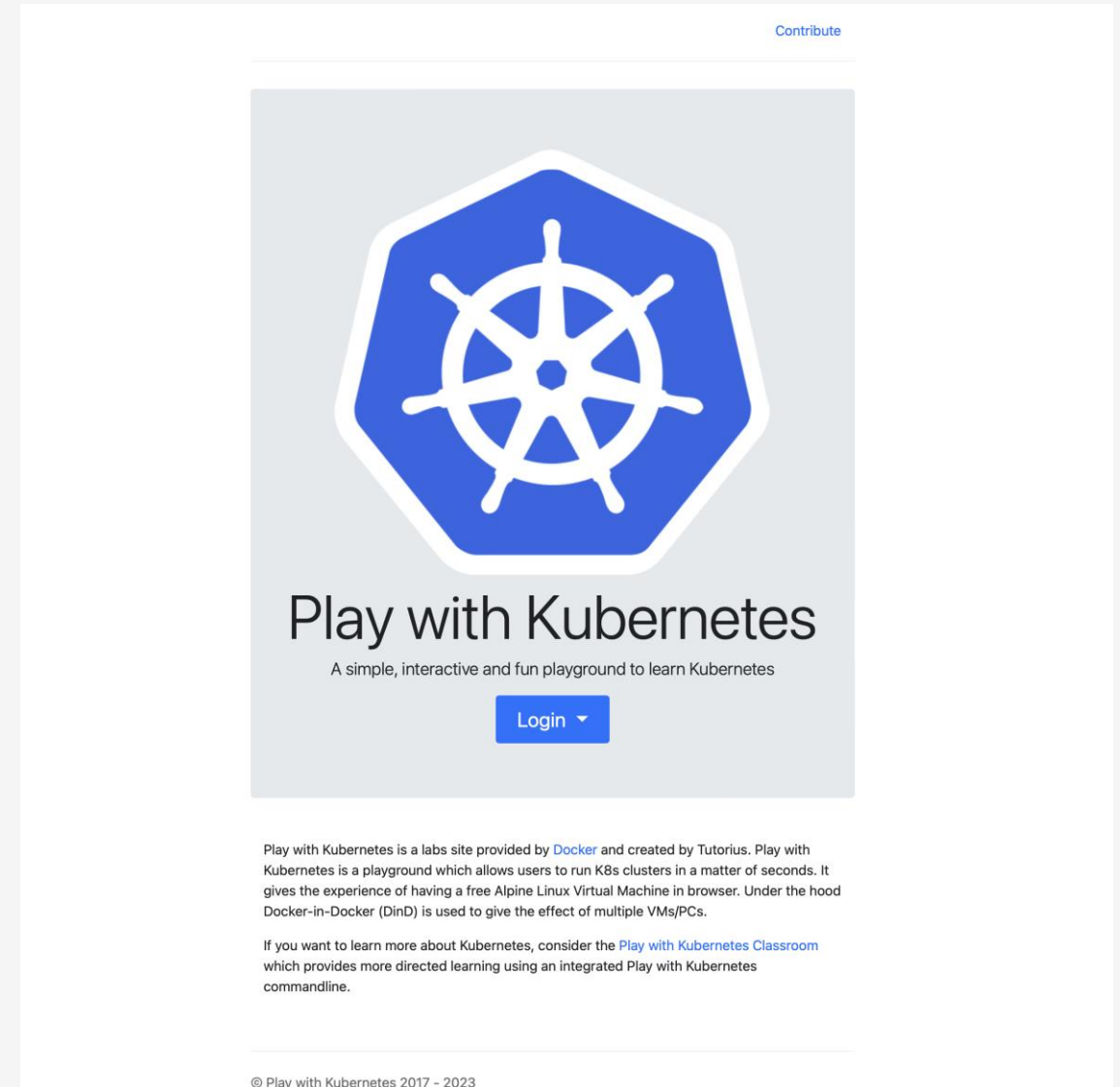
실습 진행

1. 환경 구축

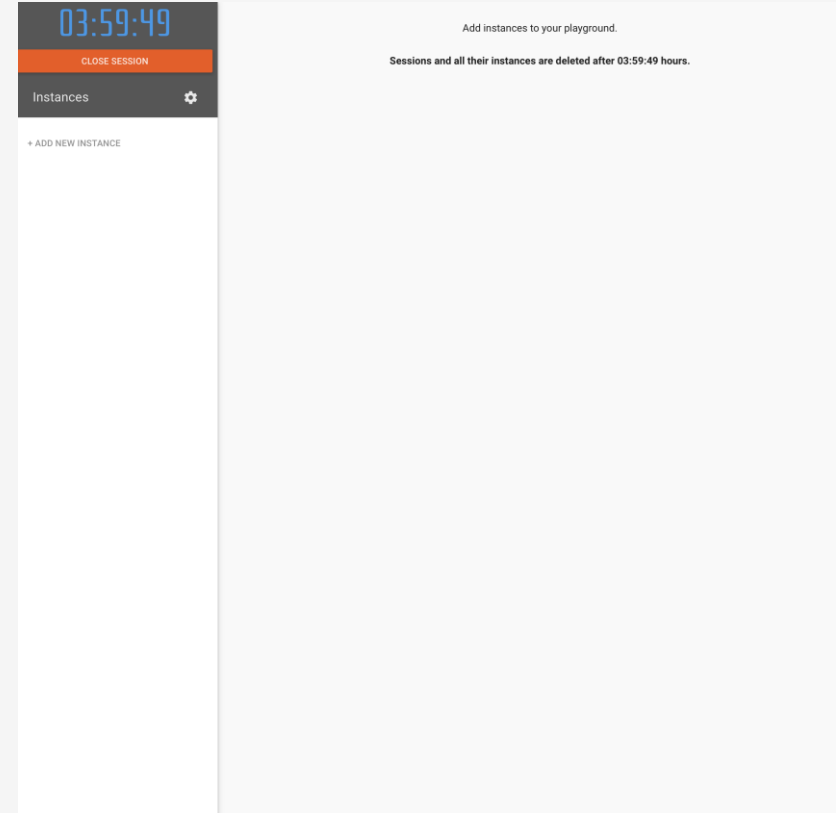
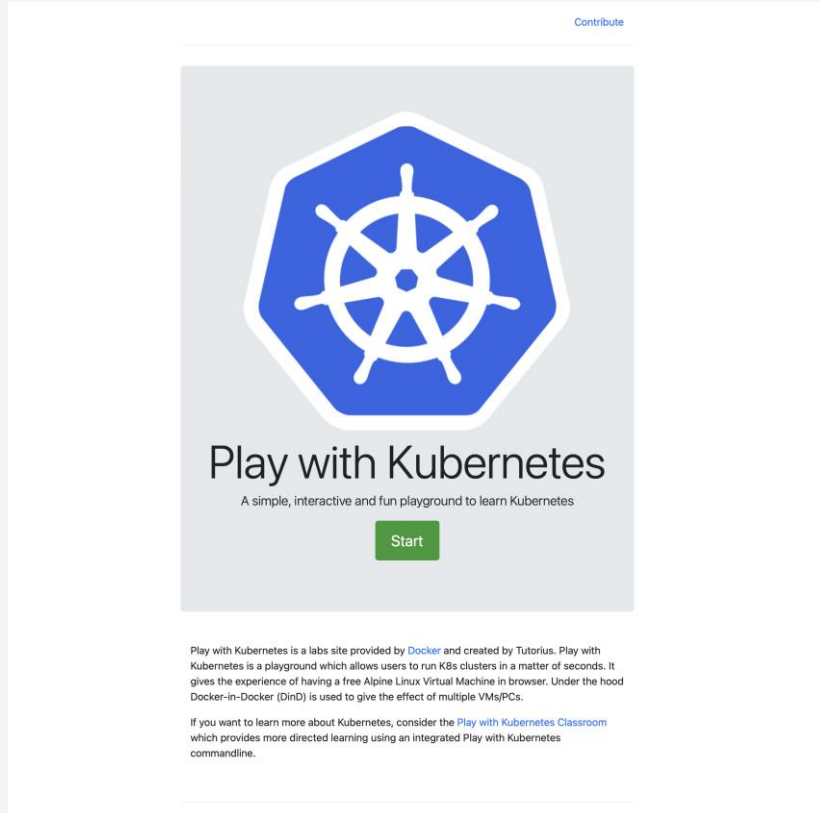
환경 구축

쿠버네티스 환경 구축

- 직접 install하지 않고, 아래 온라인 테스트 환경에서 진행
- <https://labs.play-with-k8s.com>



환경 구축



환경 구축

03:59:24

CLOSE SESSION

Instances

+ ADD NEW INSTANCE

192.168.0.18

node1

cli82etl_cli82mdl7r000b9f220

IP

192.168.0.18

Memory

1.16% (46.39MiB / 3.906GiB)

CPU

0.45%

URL

ip172-18-0-5-cli82etln7r000b9f21g.direct.labs.play-with-k8s.com

DELETE

WARNING!!!!

This is a sandbox environment. Using personal credentials is HIGHLY discouraged. Any consequences of doing so, are completely the user's responsibilities.

You can bootstrap a cluster as follows:

1. Initializes cluster master node:

```
kubeadm init --apiserver-advertise-address $(hostname -i) --pod-network-cidr 10.5.0.0/16
```

2. Initialize cluster networking:

```
kubectl apply -f https://raw.githubusercontent.com/cloudnativelabs/kube-router/master/daemonset/kubeadm-kube-router.yaml
```

3. (Optional) Create an nginx deployment:

```
kubectl apply -f https://raw.githubusercontent.com/kubernetes/website/master/content/en/examples/applications/nginx-app.yaml
```

The PWK team.

[node1 ~]\$

2. 쿠버네티스 아키텍처

쿠버네티스 주요 구성 요소

쿠버네티스 개념

- 쿠버네티스는 컨테이너화된 애플리케이션의 배포, 확장 및 운영을 자동화하기 위한 오픈 소스 시스템
- 구글에 의해 개발되었으며, CNCF(Cloud Native Computing Foundation)에 기여

쿠버네티스 주요 특징

- **자동화된 롤아웃 및 롤백**: 애플리케이션 업데이트 시 롤아웃을 자동으로 관리하고, 문제 발생 시 이전 버전으로 롤백
- **서비스 발견 및 로드 밸런싱**: 서비스를 사용하여 클러스터 내의 애플리케이션에 쉽게 접근하고, 트래픽을 자동으로 분산
- **스케일링**: 애플리케이션의 리소스 사용에 따라 자동 또는 수동으로 스케일링
- **자체 치유**: 실패한 컨테이너를 재시작하고, 건강하지 않은 컨테이너를 교체하며, 준비되지 않은 노드로부터 애플리케이션을 이전

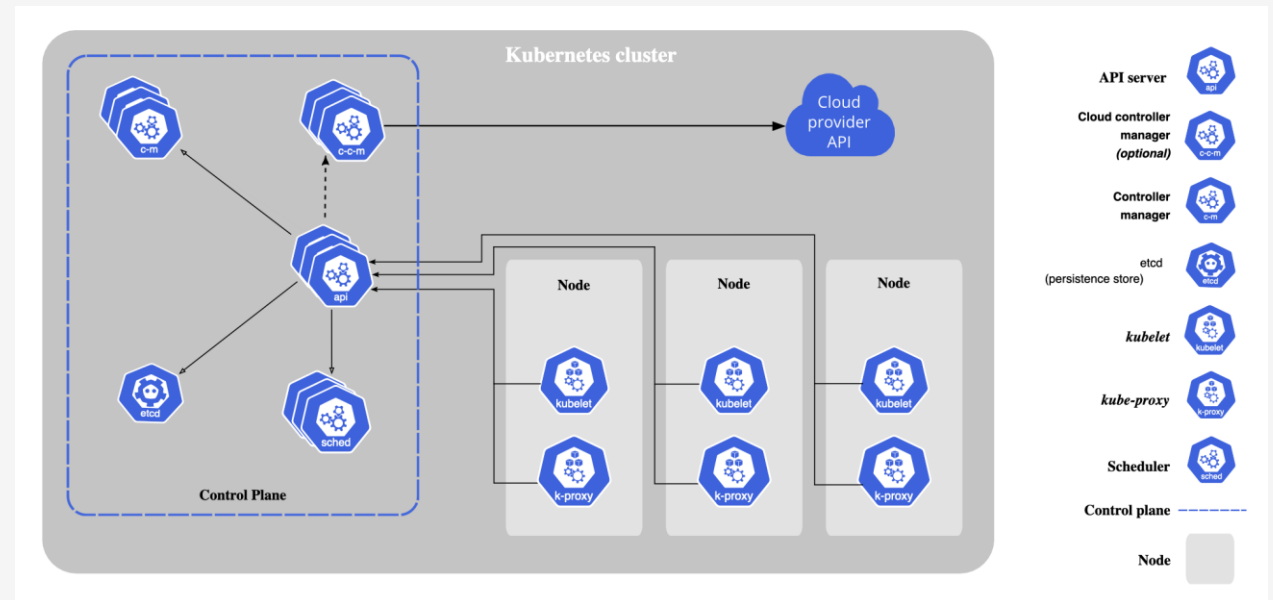
쿠버네티스 클러스터 아키텍처

쿠버네티스 아키텍처

- Master-Node 구조로 이루어진 클러스터를 사용

마스터 컴포넌트

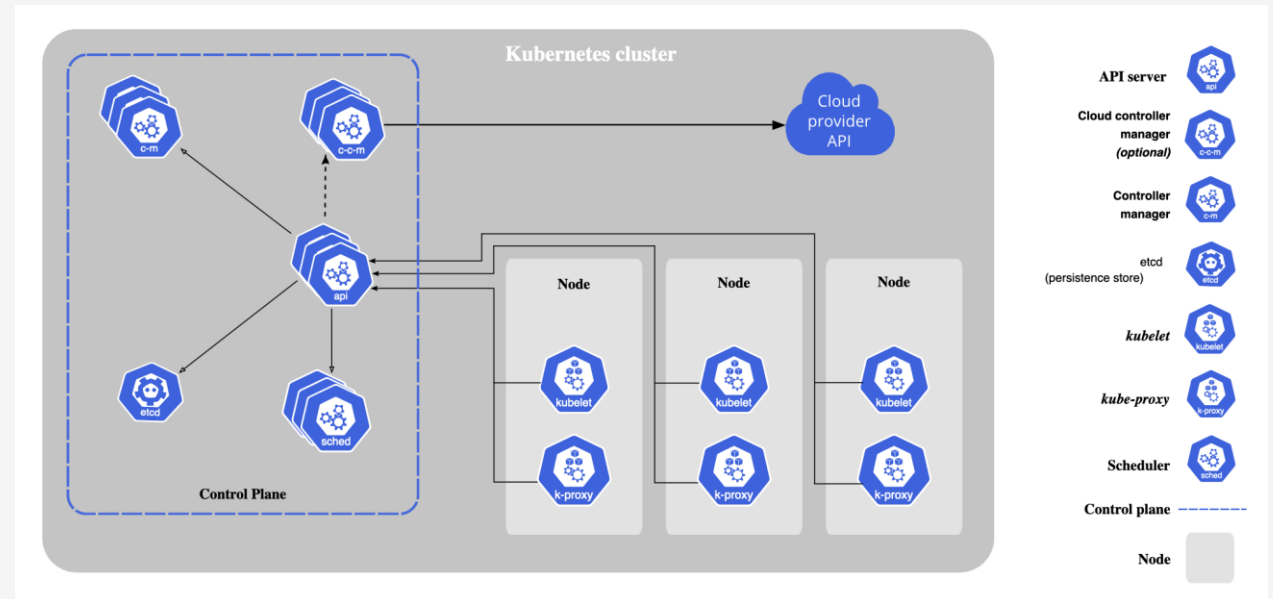
- **API 서버 (kube-apiserver):** 쿠버네티스 API를 제공하며, 사용자와 내부 컴포넌트 간의 중재자 역할
- **스케줄러 (kube-scheduler):** 새로 생성된 파드를 어떤 노드에 할당할지 결정
- **컨트롤러 매니저 (kube-controller-manager):** 여러 컨트롤러를 실행. 예를 들어, 노드 컨트롤러, 레플리케이션 컨트롤러
- **etcd:** 모든 클러스터 데이터를 저장하는 경량의 분산 키-값 저장소



쿠버네티스 클러스터 아키텍처

노드 컴포넌트

- **kubelet**: 각 노드에서 실행되며, 파드 스펙(Spec)에 설명된 대로 컨테이너가 실행되고 있는지 확인
- **kube-proxy**: 각 노드의 네트워크 규칙을 관리하여 네트워크 통신을 가능
- **컨테이너 런타임**: 컨테이너 실행을 담당하는 소프트웨어 (예: Docker, containerd, CRI-O 등).



쿠버네티스 주요 구성 요소

쿠버네티스 작동 방식

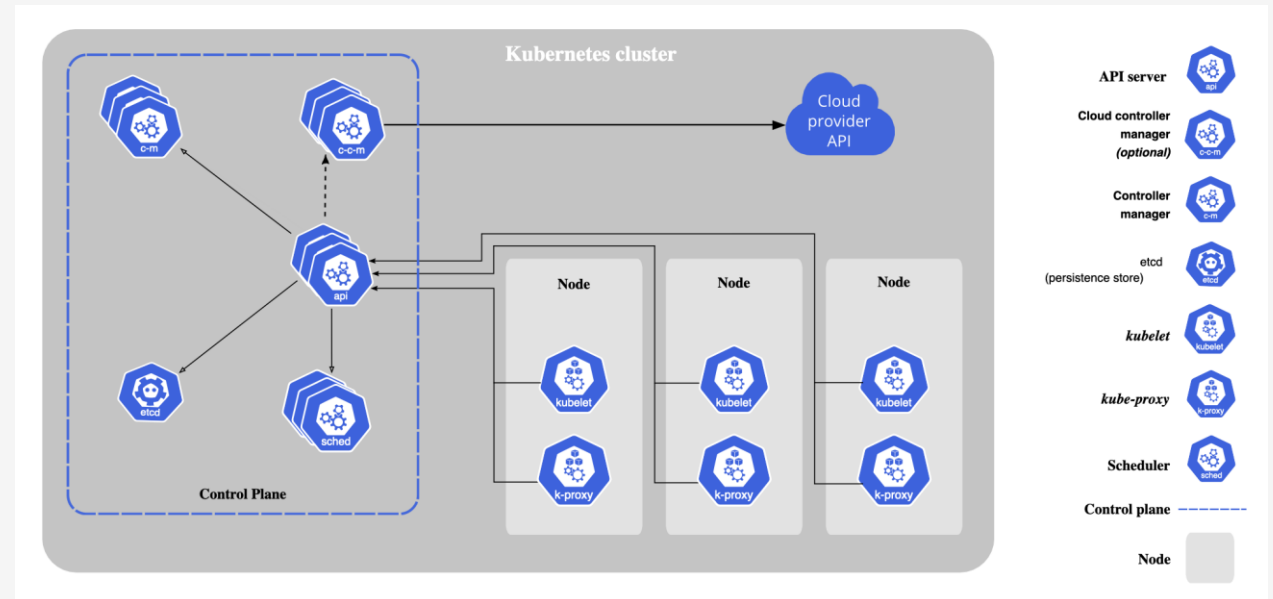
- **클러스터 구성:** 먼저 쿠버네티스 클러스터를 구성합니다. 이 클러스터는 여러 노드(물리적 또는 가상 머신)와 이들을 관리하는 마스터 노드로 구성
- **API 서버와의 통신:** 사용자는 kubectl 명령줄 인터페이스나 API를 통해 API 서버와 통신. 이를 통해 파드 생성, 업데이트, 삭제 등의 작업을 요청
- **스케줄링과 실행:** 스케줄러는 새로운 파드에 대해 가장 적합한 노드를 선택합니다. kubelet은 해당 노드에서 파드의 컨테이너가 올바르게 실행되도록 관리
- **서비스 관리:** kube-proxy는 서비스를 통한 네트워크 트래픽을 관리합니다. 서비스는 파드 그룹에 안정적인 접근점을 제공
- **자동화된 롤아웃 및 롤백:** 데플로이먼트를 통해 애플리케이션의 업데이트, 롤아웃 및 롤백을 자동으로 관리
- **스케일링과 자체 치유:** 클러스터는 애플리케이션의 수요에 따라 자동으로 스케일링하고, 실패한 파드를 재시작하는 등의 자체 치유 기능을 제공

3. 쿠버네티스 주요 구성 요소

쿠버네티스 주요 구성 요소

쿠버네티스 주요 구성 요소

- Pod
- Service
- Deployment
- ReplicaSet
- NameSpace



Pod

Pod란

- 쿠버네티스에서 배포할 수 있는 가장 작은 작업 단위
- 하나 이상의 컨테이너를 포함할 수 있으며, 이들은 스토리지와 네트워크를 공유

주요 특징

- **컨테이너 그룹핑**: 파드는 하나 이상의 밀접하게 관련된 컨테이너를 그룹화하며, 이 컨테이너들은 같은 컴퓨팅 리소스를 공유
- **공유 리소스**: 파드 내의 컨테이너는 같은 IP 주소와 포트 공간을 공유하고, 서로 localhost를 통해 통신
- **일시적인 성격**: 파드는 일시적입니다. 즉, 파드가 삭제되면 그 안의 컨테이너도 함께 삭제. 이는 파드가 불변적이지 않고 변경 가능한 리소스로 간주됨을 의미
- **생명주기**: 파드는 생성되고, 실행되며, 종료될 때까지의 생명주기를 가집니다. 파드가 종료되면, 쿠버네티스 클러스터에서 제거

사용 예

- 단일 컨테이너 파드: 대부분의 파드는 하나의 컨테이너만을 실행
- 멀티 컨테이너 파드: 로깅, 데이터 백업, 데이터 처리와 같은 보조 기능을 수행하는 사이드카(Sidecar) 컨테이너를 포함

Service

Service란

- 서비스는 파드의 집합에 대한 안정적인 네트워크 주소를 제공
- 서비스를 통해 파드 집합에 대한 접근을 관리하고, 로드 밸런싱 및 서비스 발견을 가능

주요 특징

- **안정적인 주소 제공:** 서비스는 파드 집합에 지속적으로 접근할 수 있는 안정적인 IP 주소와 포트를 제공
- **로드 밸런싱:** 서비스는 요청을 여러 파드에 분산시켜 로드 밸런싱을 수행
- **서비스 발견:** DNS 또는 환경 변수를 통해 클러스터 내의 다른 파드가 서비스를 쉽게 찾을 수 있음
- **서비스 타입:** 다양한 서비스 타입(ClusterIP, NodePort, LoadBalancer, ExternalName)을 통해 다양한 네트워크 요구 사항을 충족

사용 예

- **ClusterIP:** 클러스터 내부에서만 접근 가능한 서비스를 만들 때 사용
- **LoadBalancer:** 클라우드 제공 업체의 로드 밸런서를 사용하여 서비스에 대한 외부 접근을 관리

Deployment

Deployment란

- 데플로이먼트는 쿠버네티스에서 파드와 레플리카셋의 상태를 선언적으로 관리하는 API 오브젝트
- 애플리케이션의 배포, 업데이트, 스케일링 등을 자동화하고 관리

주요 특징

- **자동화된 롤아웃과 롤백:** 데플로이먼트는 애플리케이션의 새 버전을 롤아웃하고, 필요한 경우 이전 버전으로 롤백하는 프로세스를 자동화
- **상태 관리:** 원하는 상태(Desired State)를 정의하고, 쿠버네티스가 실제 상태(Current State)를 그 상태로 유지
- **선언적 업데이트:** YAML 파일이나 JSON 형식을 사용하여 애플리케이션을 업데이트하는 방식을 선언
- **스케일링:** 파드의 수를 수동 또는 자동으로 조절하여 애플리케이션을 스케일링

사용 예

- **새 버전 배포:** 데플로이먼트를 사용하여 애플리케이션의 새 버전을 롤아웃
- **스케일링:** `kubectl scale deployment` 명령어를 사용하여 데플로이먼트를 확장하거나 축소

ReplicaSet

ReplicaSet란?

- 레플리카셋은 파드의 복제본을 유지 관리하는 쿠버네티스 오브젝트
- 파드의 원하는 복제본 수를 지정하고, 지정된 수의 파드 복제본이 항상 실행되고 있도록 보장

주요 특징

- **복제본 수 관리:** 지정된 수의 파드 복제본을 유지
- **자체 치유:** 실패한 파드를 자동으로 대체하여 복제본 수를 유지
- **유연한 파드 선택:** 레이블 선택기(Label Selector)를 사용하여 관리할 파드를 결정

사용 예

- **애플리케이션 가용성 보장:** 레플리카셋은 애플리케이션의 가용성을 높이기 위해 여러 파드의 복제본을 실행
- **부하 분산:** 여러 파드 복제본을 통해 트래픽이 분산되어, 애플리케이션에 대한 부하가 각 복제본에 균등하게 분배

4. 실습 진행

실습 진행

WARNING!!!!

This is a sandbox environment. Using personal credentials is HIGHLY! discouraged. Any consequences of doing so, are completely the user's responsibilities.

You can bootstrap a cluster as follows:

1. Initializes cluster master node:

```
kubeadm init --apiserver-advertise-address $(hostname -i) --pod-network-cidr 10.5.0.0/16
```

2. Initialize cluster networking:

```
kubect1 apply -f https://raw.githubusercontent.com/cloudnativelabs/kube-router/master/daemonset/kubeadm-kube-router.yaml
```

3. (Optional) Create an nginx deployment:

```
kubect1 apply -f https://raw.githubusercontent.com/kubernetes/website/master/content/en/examples/application/nginx-app.yaml
```

The PWK team.

실습 진행

pod 실습 진행

- `kubectl run fastcampusserver --image=nginx:latest --port 80`
- `kubectl get pods`
- `kubectl describe pod`

```
[node1 ~]$ kubectl get pods
NAME                READY   STATUS    RESTARTS   AGE
fastcampusserver    0/1     Pending   0           107s

[node1 ~]$ kubectl describe pod
Name:                fastcampusserver
Namespace:           default
Priority:             0
Service Account:     default
Node:                <none>
Labels:              run=fastcampusserver
Annotations:         <none>
Status:              Pending
IP:                  <none>
IPs:                 <none>
Containers:
  fastcampusserver:
    Image:            nginx:latest
    Port:             80/TCP
    Host Port:        0/TCP
    Environment:      <none>
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-hvb9r (ro)
Conditions:
  Type           Status
  PodScheduled   False
Volumes:
  kube-api-access-hvb9r:
    Type:              Projected (a volume that contains injected data from multiple sources)
    TokenExpirationSeconds: 3607
    ConfigMapName:      kube-root-ca.crt
    ConfigMapOptional:  <nil>
    DownwardAPI:        true
QoS Class:           BestEffort
Node-Selectors:      <none>
Tolerations:         node.kubernetes.io/not-ready:NoExecute op=Exists for 300s
                     node.kubernetes.io/unreachable:NoExecute op=Exists for 300s
Events:
  Type     Reason             Age   From                    Message
  ----     -
  Warning   FailedScheduling   112s  default-scheduler       0/1 nodes are available: 1 node(s) had untolerated taint {node-role.kubernetes.io/control-plane: }. Preemption is not helpful for scheduling..
```

실습 진행

pod 실습 진행

- `kubectl run fastcampusserver --image=nginx:latest --port 80`
- `kubectl get pods`
- `kubectl describe pod`

```
[node1 ~]$ kubectl describe pod
Name:          fastcampusserver
Namespace:     default
Priority:       0
Service Account: default
Node:          node2/192.168.0.17
Start Time:    Sat, 02 Dec 2023 01:39:45 +0000
Labels:        run=fastcampusserver
Annotations:    <none>
Status:        Running
IP:            10.5.1.3
IPs:
  IP: 10.5.1.3
Containers:
  fastcampusserver:
    Container ID:   containerd://a4fa36ec0cef4bb64fc3f3d6ab4fbb59cdd8874de21e365f518e0b543d5f79b6
    Image:          nginx:latest
    Image ID:       docker.io/library/nginx@sha256:10d1f5b58f74683ad34eb29287e07dab1e90f10af243f151bb50aa5dbb4d62ee
    Port:           80/TCP
    Host Port:      0/TCP
    State:          Running
      Started:      Sat, 02 Dec 2023 01:40:16 +0000
    Ready:          True
    Restart Count:   0
    Environment:    <none>
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-hvb9r (ro)
Conditions:
  Type            Status
  Initialized      True
  Ready            True
  ContainersReady  True
  PodScheduled     True
Volumes:
  kube-api-access-hvb9r:
    Type:          Projected (a volume that contains injected data from multiple sources)
    TokenExpirationSeconds: 3607
    ConfigMapName:    kube-root-ca.crt
    ConfigMapOptional: <nil>
    DownwardAPI:      true
QoS Class:         BestEffort
Node-Selectors:    <none>
```

실습 진행

pod 실습 진행

- curl [IP Address]:80

```
[node1 ~]$ curl 10.5.1.3:80
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
```

실습 진행

deployment 실습 진행

- `kubectl create deployment fastcampusdeployment --image=nginx --replicas=2`
- `kubectl get deployments.apps`
- `kubectl get pods`
- `kubectl describe pod [pod name]`
- `kubectl edit deployments.apps fastcampusdeployment`
- `kubectl get pods`